

به نام خدا



دانشگاه تهران



دانشکده مهندسی برق و کامپیوتر

درس هوش مصنوعی در سیستم های نهان

تمرین اول

نام و نام خانوادگی	بهزاد جنتی
شماره دانشجویی	۸۱۰۱۰۳۰۹۸
استاد	دکتر مهدی مدرسی / دکتر مصطفی صالحی ارسالی
تاریخ ارسال گزارش	۱۴۰۴.۱۰.۱۹

فهرست

سوال ۱ - پیاده‌سازی توابع پایه و اعتبارسنجی سیستم LNS.....	۵
۱-۱. مقدمه و بیان مسئله.....	۵
۱-۲. مبانی ریاضی و فرمول‌بندی.....	۵
۱-۳. شرح پیاده‌سازی (Implementation Details).....	۶
۱-۴. نتایج شبیه‌سازی و اعتبارسنجی.....	۷
۱-۵. تحلیل نتایج.....	۸
سوال ۲: تحلیل خطای تبدیل با پارامترهای مختلف در سیستم LNS.....	۹
۲-۱. مقدمه و تعریف مسئله.....	۹
۲-۲. مبانی ریاضی و فرمولاسیون.....	۹
۲-۳. شرح پیاده‌سازی و آزمایش.....	۱۰
۲-۴. نتایج و تحلیل نمودارها.....	۱۲
تحلیل و تفسیر نتایج (Trade-off Analysis).....	۱۲
سوال ۳: عملیات جمع در حوزه LNS و روش‌های تقریب.....	۱۵
۳-۱. مقدمه و چالش جمع.....	۱۵
۳-۲. روش‌های پیاده‌سازی شده.....	۱۶
۳-۳. نتایج آزمایش و اعتبارسنجی.....	۱۷
۳-۴. تحلیل نتایج و مقایسه روش‌ها.....	۱۷
تحلیل نتایج.....	۱۹
سوال ۴: تحلیل آماری خطای جمع LNS با داده‌های تصادفی.....	۱۹
۴-۱. مقدمه و هدف آزمایش.....	۱۹
۴-۲. متدولوژی و تنظیمات سیستم.....	۲۰
۴-۳. تحلیل نتایج آماری.....	۲۱
۴-۴. تحلیل رفتار خطا نسبت به اختلاف توان‌ها (d).....	۲۱
تحلیل نمودارها و داده‌های دسته‌بندی شده:.....	۲۲
۴-۵. تحلیل اثر سایز LUT بر دقت.....	۲۳

۴-۶. نتیجه‌گیری نهایی بخش چهارم	۲۴
سوال ۵: تحلیل تأثیر انتخاب مبنا بر بازه دینامیکی و خطای سیستم LNS	۲۴
۵-۱. مقدمه و بیان مسئله	۲۴
۵-۲. مبانی نظری و فرمولاسیون	۲۵
۵-۳. شرح آزمایش	۲۵
۵-۴. نتایج شبیه‌سازی و تحلیل آماری	۲۶
۵-۵. بحث و نتیجه‌گیری (The Cost of Range)	۲۸
سوال ۶: پیاده‌سازی و تحلیل لایه کانولوشن شبکه عصبی در حوزه LNS	۲۹
۶-۱. مقدمه و هدف	۲۹
۶-۲. پیکربندی آزمایش	۳۰
۶-۴. تحلیل نتایج تجربی	۳۱
بحث و چالش‌های پیاده‌سازی (Accumulation Challenge)	۳۲
۶-۵. تحلیل عمیق و بحث (چرا خطاها زیاد است؟)	۳۳
۶-۶. نتیجه‌گیری نهایی پروژه	۳۳

شکل‌ها

- شکل ۱- کلاس LNSNumber تعریف شده در کد پایتون ۶
- شکل ۲- خروجی گرفته شده برای سوال ۱ بعد از اجرای کد پایتون ۷
- شکل ۳- محاسبه خطا برای سوال ۲ با استفاده از توابع نوشته شده سوال ۱ در کد پایتون ۱۰
- شکل ۴- تابع آنالیز پارامترها برای سوال ۲ در کد پایتون ۱۱
- شکل ۵- پارامترهای ست شده برای سوال ۲ ۱۱
- شکل ۶- نمودار trade off بین رنج دینامیک و دقت برای مبناهای مختلف ۱۳
- شکل ۷- خروجی کد پایتون برای Dynamic Range ۱۳
- شکل ۸- نمودار های مقایسه خطای مطلق و نسبی با توجه به سائز ورودی ۱۴
- شکل ۹- تصویر trade off بین مبنا با خطا و رنج ۱۵
- شکل ۱۰- تابع نوشته شده برای عملیات جمع با روش LUT ۱۷
- شکل ۱۱- مقایسه دو روش LUT با Shift ۱۸
- شکل ۱۲- تنظیم پارامتر های مسئله برای سوال ۴ در کد پایتون ۲۰
- شکل ۱۳- نمودار تاثیر سائز LUT بر میانگین ارور نسبی آن ۲۳
- شکل ۱۴- پارامترها و مبناهای مورد استفاده در سوال ۵ در کد پایتون ۲۶
- شکل ۱۵- تاثیر مبنا در رنج دینامیک با تعداد بیت یکسان ۲۷
- شکل ۱۶- نمودار تاثیر مبنا در خطای کوانتیزیشن ۲۸

جدول‌ها

- جدول ۱- نتایج تبدیل اعداد تست و خطاهای محاسبه شده..... ۸
- جدول ۲- خلاصه نتایج سوال ۲ (محاسبه خطا)..... ۱۲
- جدول ۳- مقایسه روش‌های جمع LNS و خطای تقریب‌ها..... ۱۷
- جدول ۴- خلاصه آماری خطاهای نسبی در ۱۰۰۰ نمونه تست..... ۲۱
- جدول ۵- تاثیر سایز LUT بر خطای میانگین..... ۲۳
- جدول ۶- مقایسه بازه دینامیکی به ازای مبناهای مختلف..... ۲۶
- جدول ۷- مقایسه خطای کوانتیزیشن..... ۲۷
- جدول ۸- مقایسه خروجی کانولوشن در روش‌های مختلف..... ۳۱
- جدول ۹- آمار خطا در ۱۰ تکرار..... ۳۲

سوال ۱ – پیاده‌سازی توابع پایه و اعتبارسنجی سیستم LNS

۱-۱. مقدمه و بیان مسئله

در محاسبات دیجیتال، به‌ویژه در سیستم‌های نهان (Embedded Systems) که با محدودیت‌های توان و منابع سخت‌افزاری روبرو هستند، بهینه‌سازی عملیات ریاضی از اهمیت بالایی برخوردار است. سیستم‌های متداول مانند ممیز شناور (Floating-Point)، علیرغم دقت بالا، هزینه محاسباتی زیادی برای عملیات ضرب و تقسیم دارند. در مقابل، سیستم اعداد لگاریتمی (Logarithmic Number System - LNS) راهکاری جایگزین ارائه می‌دهد.

در LNS، هر عدد حقیقی X به صورت لگاریتم قدر مطلق آن ذخیره می‌شود. این ویژگی باعث می‌شود که عملیات ضرب و تقسیم که در حوزه اعداد حقیقی پرهزینه هستند، به عملیات جمع و تفریق در حوزه لگاریتمی تبدیل شوند:

$$A \times B \Leftrightarrow \log(A) + \log(B)$$

$$A/B \Leftrightarrow \log(A) - \log(B)$$

هدف اصلی این بخش، پیاده‌سازی نرم‌افزاری ساختار داده LNS، توابع تبدیل دوطرفه از حقیقی به LNS و برعکس (و بررسی دقت و خطای کوانتیزیشن در این سیستم) است.

۱-۲. مبانی ریاضی و فرمول‌بندی

یک عدد حقیقی X در سیستم LNS پیشنهادی با سه‌تایی مرتب (z_x, s_x, m_x) نمایش داده می‌شود:

۱. **پرچم صفر** (z_x): از آنجا که $\log(0) = -\infty$ است، عدد صفر قابل نمایش با فرمت عادی نیست. بنابراین، اگر $z_x = 1$ باشد، عدد برابر با صفر مطلق در نظر گرفته می‌شود.

۲. **بیت علامت** (s_x): نشان‌دهنده علامت عدد اصلی است (۰ برای مثبت و ۱ برای منفی).

۳. **مانتیس** (m_x): مقدار لگاریتم عدد که به صورت ممیز ثابت (Fixed-Point) ذخیره می‌شود.

رابطه تبدیل مقدار LNS به مقدار حقیقی به صورت زیر است:

$$X = (-1)^{s_x} \cdot (1 - z_x) \cdot \text{base}^{(m_x \cdot 2^{-f})}$$

که در آن:

- W : عرض بیت کل مانتیس (Word Length)
- f : تعداد بیت‌های بخش اعشاری (Fractional Bits)
- m_x : یک عدد صحیح علامت‌دار (Two's Complement) با طول W بیت است.

مقدار m_x از رابطه زیر محاسبه و گرد می‌شود:

$$m_x = \text{round}(\log_{\text{base}}(|X|) \times 2^f)$$

همچنین مقدار m_x باید در بازه قابل نمایش $[-2^{W-1}, 2^{W-1} - 1]$ محدود (Clamp) شود.

۱-۳. شرح پیاده‌سازی (Implementation Details)

برای پیاده‌سازی این سیستم در پایتون، کلاس LNSNumber طراحی شد که ویژگی‌های z, s, m را به همراه پارامترهای سیستم (W, f, base) نگهداری می‌کند.

```

LNS (Logarithmic Number System) Conversion Functions

[ ] ▶ import numpy as np
    from typing import Tuple, List
    import warnings
    warnings.filterwarnings('ignore')

    class LNSNumber:
        """
        Represents a number in Logarithmic Number System (LNS).

        Format: (z, s, m) where:
        - z (bool): Zero flag (True if number is zero)
        - s (bool): Sign flag (True if negative)
        - m (int): Fixed-point representation of log_base(|x|) in two's complement
        """

        def __init__(self, z: bool, s: bool, m: int, base: float, W: int, f: int):
            """
            Initialize an LNS number.

            Args:
            z: Zero flag
            s: Sign flag (True for negative)
            m: Fixed-point mantissa (log value in fixed-point)
            base: Logarithm base
            W: Total bit width for mantissa
    
```

شکل ۱- کلاس LNSNumber تعریف شده در کد پایتون

سه تابع اصلی زیر توسعه داده شدند:

۱. **real_to_lns**: این تابع ابتدا بررسی می‌کند اگر عدد صفر باشد، پرچم z را فعال می‌کند. در غیر این صورت، لگاریتم عدد در مبنای مشخص شده محاسبه شده، در 2^f ضرب می‌شود و به نزدیک‌ترین عدد صحیح گرد می‌شود تا مقدار m_x بدست آید. همچنین سرریز (Overflow) و پاریز (Underflow) با محدود کردن مقدار m به ماکزیمم و مینییم بازه اعداد صحیح W بیتی مدیریت می‌شود.

۲. **lns_to_real**: این تابع فرآیند معکوس را انجام می‌دهد. مقدار صحیح m_x را بر 2^f تقسیم کرده تا مقدار واقعی لگاریتم بدست آید و سپس آن را به توان پایه می‌رساند.

۳. **get_lns_range**: کمترین و بیشترین عدد مثبت قابل نمایش را محاسبه می‌کند.

$$X_{min} = \text{base}^{-2^{W-1} \times 2^{-f}}$$

$$X_{max} = \text{base}^{(2^{W-1}-1) \times 2^{-f}}$$

۴-۱. نتایج شبیه‌سازی و اعتبارسنجی

طبق صورت پروژه، پارامترهای سیستم برای تست اولیه به صورت زیر تنظیم شدند:

- مبنا (Base): ۲
- طول کلمه (W): ۶ بیت
- بخش اعشار (f): ۳ بیت

(تحلیل بازه قابل نمایش): با این تنظیمات، بخش صحیح مانیتیس $W - f = 3$ بیت خواهد بود.

- بازه مقادیر صحیح m : از -32 تا $+31$.
- بازه مقادیر لگاریتمی: از -4.0 تا $+3.875$.
- بازه اعداد حقیقی مثبت قابل نمایش:

$$[2^{-4}, 2^{3.875}] \approx [0.0625, 14.672]$$

*** =====				
PART 1: LNS Conversion Functions Validation				
=====				
Parameters: base=2, W=6 (total bits), f=3 (fractional bits)				
Integer part bits: W - f = 3 bits				

Representable Range for Positive Numbers:				

Mantissa integer range: [-32, 31]				
Log value range: [-4.0000, 3.8750]				
Positive real range: [0.062500, 14.672065]				
In scientific notation: [6.2500e-02, 1.4672e+01]				

Conversion Results Table:				

Original	LNS (z,s,m)	Converted	Abs Error	Rel Error
0.0000	(1, -, -)	0.000000	0.000000	0.000000
1.0000	(0, 0, 0)	1.000000	0.000000	0.000000
1.5000	(0, 0, 5)	1.542211	0.042211	0.028141
0.2500	(0, 0, -16)	0.250000	0.000000	0.000000
-2.0000	(0, 1, 8)	-2.000000	0.000000	0.000000
12.0000	(0, 0, 29)	12.337687	0.337687	0.028141
15.0000	(0, 0, 31)	14.672065	0.327935	0.021862
-7.5000	(0, 1, 23)	-7.336032	0.163968	0.021862
0.0010	(0, 0, -32)	0.062500	0.061500	61.500000

شکل ۲- خروجی گرفته شده برای سوال ۱ بعد از اجرای کد پایتون

در ادامه و صفحه بعد جدول نتایج گرفته شده به شکل کامل قرار داده شده است.

جدول 1 - نتایج تبدیل اعداد تست و خطاهای محاسبه شده

خطای نسبی (Rel Err)	خطای مطلق (AbsErr)	مقدار بازگشتی (Converted)	نمایش LNS (z, s, m)	عدد اصلی (Original)
0.000000	0.000000	0.000000	(1, -, -)	0.0
0.000000	0.000000	1.000000	(0, 0, 0)	1.0
0.028141	0.042211	1.542211	(0, 0, 5)	1.5
0.000000	0.000000	0.250000	(0, 0, -16)	0.25
0.000000	0.000000	-2.000000	(0, 1, 8)	-2.0
0.028141	0.337687	12.337687	(0, 0, 29)	12.0
0.021862	0.327935	14.672065	(0, 0, 31)	15.0
0.021862	0.163968	-7.336032	(0, 1, 23)	-7.5
61.500000	0.061500	0.062500	(0, 0, -32)	0.001

۱-۵. تحلیل نتایج

با بررسی جدول فوق، نکات تحلیلی زیر قابل استخراج است:

۱. **دقت کامل در توان‌های مبنا:** اعداد $1.0(2^0)$ ، $0.25(2^{-2})$ و $-2.0(-2^1)$ که دقیقاً توان‌های صحیحی از مبنا (عدد ۲) هستند، با **خطای صفر** نمایش داده شده‌اند. این ویژگی ذاتی سیستم LNS است که برای توان‌های مبنا دقتی معادل ممیز شناور دارد.
۲. **خطای کوانتیزیشن در اعداد غیر توان ۲:** اعدادی مانند 1.5 و 12.0 دارای خطا هستند. دلیل این امر این است که $\log_2(1.5) \approx 0.585$ است، اما با ۳ بیت اعشار، نزدیکترین مقادیر قابل نمایش 0.500 (برای $m = 4$) و 0.625 (برای $m = 5$) هستند. سیستم مقدار 0.625 را انتخاب کرده که باعث ایجاد خطای نسبی حدود 2.8% شده است.
۳. **محدودیت بازه دینامیکی (Dynamic Range):** عدد 0.001 خطای بسیار فاحشی (۶۱۵۰ درصد!) دارد.
 - مقدار واقعی: $\log_2(0.001) \approx -9.96$.
 - کمترین مقدار قابل نمایش سیستم: -4.0. سیستم به ناچار عدد را به کمترین مقدار ممکن (Clamping به $m = -32$) محدود کرده است که باعث شده عدد 0.001 به 0.0625 تبدیل شود. این مورد ضعف اصلی LNS با تعداد بیت کم را در پوشش اعداد بسیار کوچک نشان می‌دهد.
۴. **رفتار در کران بالا:** عدد 15.0 نیز به 14.67 تبدیل شده است، زیرا $\log_2(15) \approx 3.9$ است اما ماکزیمم مقدار لگاریتمی قابل نمایش 3.875 می‌باشد. با این حال، خطای اشباع در اینجا (حدود ۲%) بسیار کمتر از خطای Underflow در مورد عدد 0.001 است.

سوال ۲: تحلیل خطای تبدیل با پارامترهای مختلف در سیستم

LNS

۲-۱. مقدمه و تعریف مسئله

همان طور که در بخش قبل هم گفته شد در سیستم نمایش اعداد لگاریتمی (LNS)، دقت نمایش و بازه دینامیکی (Dynamic Range) به شدت وابسته به سه پارامتر کلیدی است:

۱. مبنا (b): پایه لگاریتم.

۲. عرض بیت کل (W): تعداد کل بیت‌های اختصاص داده شده به مانتیس (بدون در نظر گرفتن بیت‌های پرچم صفر و علامت).

۳. تعداد بیت‌های اعشار (f): تعداد بیت‌هایی که برای نمایش بخش اعشاری لگاریتم استفاده می‌شوند.

هدف این بخش، بررسی تاثیر تغییر همزمان این پارامترها بر روی دقت (میزان خطا) و بازه قابل نمایش است. ما چهار پیکربندی مختلف را بررسی می‌کنیم تا مصالحه (Trade-off) بین کاهش عرض بیت (W) و افزایش مبنا (b) را تحلیل کنیم.

۲-۲. مبانی ریاضی و فرمولاسیون

در سیستم LNS، مقدار واقعی X از طریق رابطه زیر به دست می‌آید:

$$X \approx b^{(m \cdot 2^{-f})}$$

که در آن m یک عدد صحیح علامت‌دار در نمایش مکمل دو (Two's Complement) با طول

W بیت است. بنابراین، بازه مقادیر صحیح m عبارت است از:

$$-2^{W-1} \leq m \leq 2^{W-1} - 1$$

الف) محاسبه بازه قابل نمایش (Dynamic Range)

بازه اعداد مثبت قابل نمایش $[X_{min}, X_{max}]$ به صورت زیر محاسبه می‌شود:

$$X_{min} = b^{\frac{-2^{W-1}}{2^f}}$$
$$X_{max} = b^{\frac{2^{W-1}-1}{2^f}}$$

ب) محاسبه خطا

برای هر عدد حقیقی x ، مقدار تقریبی آن در LNS ($\hat{x}$) محاسبه شده و خطاهای زیر استخراج می‌شوند:

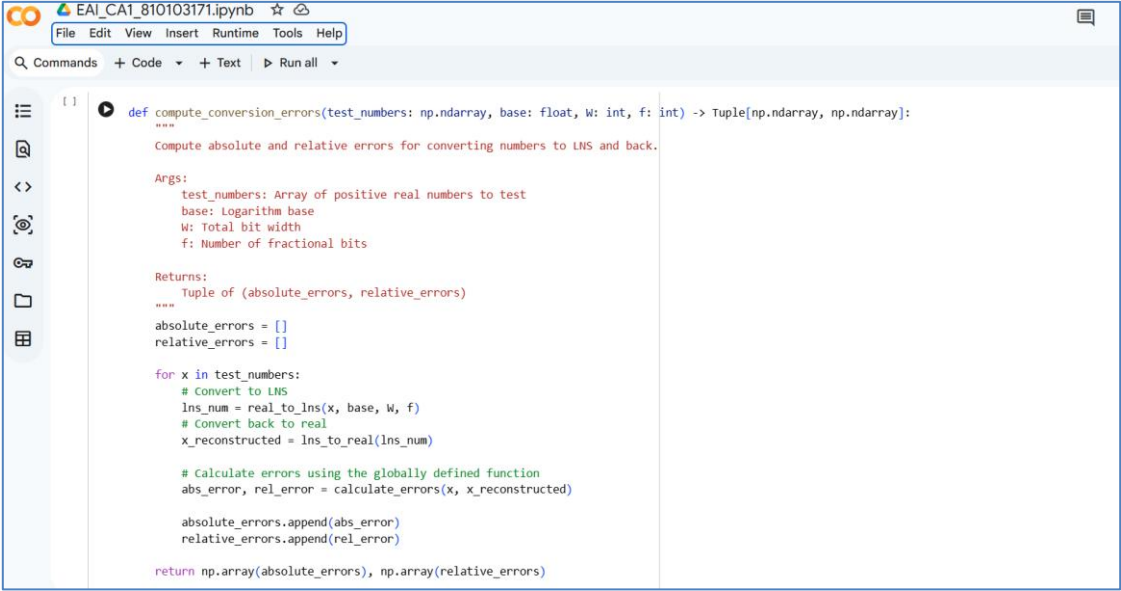
۱. خطای مطلق (Absolute Error):

$$E_{abs} = |x - \hat{x}|$$

۲. خطای نسبی (Relative Error):

$$E_{rel} = \frac{|x - \hat{x}|}{|x|}$$

برای محاسبه خطا در این بخش از تابع نوشته شده در کلاس LNS سوال ۱ به شکل زیر استفاده می‌کنیم:



```
def compute_conversion_errors(test_numbers: np.ndarray, base: float, W: int, f: int) -> Tuple[np.ndarray, np.ndarray]:
    """
    Compute absolute and relative errors for converting numbers to LNS and back.

    Args:
        test_numbers: Array of positive real numbers to test
        base: Logarithm base
        W: Total bit width
        f: Number of fractional bits

    Returns:
        Tuple of (absolute_errors, relative_errors)
    """
    absolute_errors = []
    relative_errors = []

    for x in test_numbers:
        # Convert to LNS
        lns_num = real_to_lns(x, base, W, f)
        # Convert back to real
        x_reconstructed = lns_to_real(lns_num)

        # Calculate errors using the globally defined function
        abs_error, rel_error = calculate_errors(x, x_reconstructed)

        absolute_errors.append(abs_error)
        relative_errors.append(rel_error)

    return np.array(absolute_errors), np.array(relative_errors)
```

شکل ۳- محاسبه خطا برای سوال ۲ با استفاده از توابع نوشته شده سوال ۱ در کد پایتون

۲-۳. شرح پیاده‌سازی و آزمایش

برای تحلیل اثر پارامترها، چهار مجموعه پارامتر زیر (طبق صورت سوال) انتخاب شده‌اند:

۱. Set A: (Base= $\sqrt{2}$, $W = 8$, $f = 4$)

۲. Set B: (Base=2, $W = 7$, $f = 4$)

۳. Set C: (Base=4, $W = 6$, $f = 4$)

۴. Set D: (Base=16, $W = 5$, $f = 4$)

در پیاده سازی پایتون هم یک تابع نوشته شده که آنالیز رو برای پارامترهای ست شده در تمرین انجام میدهد که تصویر آن را در زیر میتونید مشاهده کنید:

```
def analyze_parameter_set(base: float, W: int, f: int, num_samples: int = 1000) -> Dict:
    """
    Perform complete error analysis for a given parameter set.

    Args:
        base: Logarithm base
        W: Total bit width
        f: Number of fractional bits
        num_samples: Number of test samples to generate

    Returns:
        Dictionary containing analysis results
    """
    # Get representable range using the globally defined function
    min_val, max_val = get_lns_range(base, W, f)

    # Generate uniformly distributed test numbers
    test_numbers = np.linspace(min_val, max_val, num_samples)

    # Compute errors
    abs_errors, rel_errors = compute_conversion_errors(test_numbers, base, W, f)

    # Calculate statistics
    results = {
        'base': base,
        'W': W,
        'f': f,
        'min_val': min_val,
        'max_val': max_val,
```

شکل ۴- تابع آنالیز پارامترها برای سوال ۲ در کد پایتون

در ادامه تصویر خود پارامترهای تعریف شده در کد پایتون هم قرار داده شده است:

```
# Main Analysis

# Define parameter sets as specified in the problem
parameter_sets = [
    (np.sqrt(2), 8, 4), # base=√2, W=8, f=4
    (2, 7, 4),          # base=2, W=7, f=4
    (4, 6, 4),          # base=4, W=6, f=4
    (16, 5, 4),         # base=16, W=5, f=4
]
```

شکل ۵- پارامترهای ست شده برای سوال ۲

روند انجام آزمایش (متدولوژی کد):

۱. تابع `get_lns_range` برای هر مجموعه پارامتر فراخوانی شد تا X_{min} و X_{max} دقیق محاسبه شود.

۲. تعداد ۱۰۰۰ نمونه تصادفی با توزیع یکنواخت در بازه $[X_{min}, X_{max}]$ تولید شد.

۳. هر عدد با استفاده از تابع `real_to_lns` به فرمت LNS تبدیل و سپس با `lns_to_real` به حوزه حقیقی بازگردانده شد.

۴. خطاهای مطلق و نسبی برای تمام نمونه‌ها محاسبه و میانگین‌گیری شد.

۲-۴. نتایج و تحلیل نمودارها

در این بخش، خروجی‌های حاصل از اجرای کد برای هر چهار حالت بررسی می‌شود.

جدول ۲ - خلاصه نتایج سوال ۲ (محاسبه خطا)

پیکربندی (Base, W, f)	بازه نمایش (X_{min}, X_{max})	نسبت بازه (Dynamic Range)	میانگین خطای نسبی	حداکثر خطای نسبی
$(\sqrt{2}, 8, 4)$	[0.0625, 15.65]	2.51×10^2	0.54%	1.08%
(2, 7, 4)	[0.0625, 15.32]	2.45×10^2	1.07%	2.18%
(4, 6, 4)	[0.0625, 14.67]	2.35×10^2	2.16%	4.42%
(16, 5, 4)	[0.0625, 13.45]	2.15×10^2	4.32%	9.03%

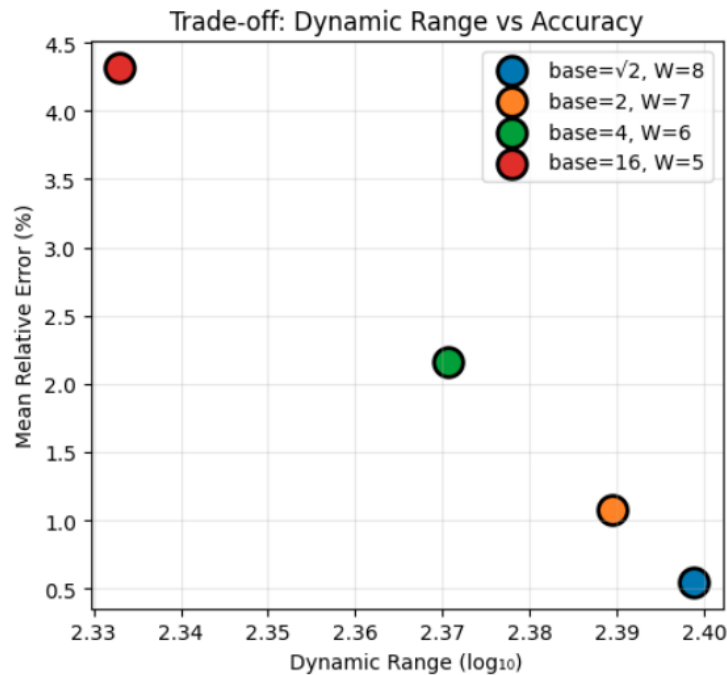
تحلیل و تفسیر نتایج (Trade-off Analysis)

با مشاهده جدول بالا و نمودارهای استخراج شده، نکات بسیار مهمی در طراحی سیستم‌های LNS آشکار می‌شود:

۱. رابطه معکوس دقت و مبنا (در عرض بیت کاهشی)

در این آزمایش، همزمان با افزایش مبنا (از $\sqrt{2}$ به 16)، تعداد بیت‌های W کاهش یافته است (از ۸ به ۵). مشاهده می‌شود که میانگین خطای نسبی به شدت افزایش یافته است (از حدود ۰.۵٪ به ۴.۳٪).

- **دلیل:** دقت در LNS متناسب با گام کوانتایزیشن است که برابر با $b^{2^{-f}}$ می‌باشد. وقتی b بزرگ می‌شود، فاصله بین اعداد قابل نمایش افزایش می‌یابد و دقت کاهش پیدا می‌کند. در اینجا چون f ثابت است ($f = 4$)، تنها عامل تعیین‌کننده رزولوشن، مبنای b است.



شکل ۶- نمودار trade off بین رنج دینامیک و دقت برای مبناهای مختلف

۲. حفظ تقریبی بازه دینامیکی (Dynamic Range)

نکته جالب این آزمایش این است که علی‌رغم کاهش بیت‌ها از ۸ به ۵، بازه دینامیکی تغییر فاحشی نکرده است (همگی حدوداً در بازه [0.06, 15] هستند).

```
***
=====
Analyzing: base=1.4142, W=8, f=4
=====
Representable Range: [6.250000e-02, 1.565715e+01]
Dynamic Range Ratio: 2.51e+02
Mean Absolute Error: 4.254620e-02
Mean Relative Error: 0.005435 (0.5435%)
Max Relative Error: 0.010883 (1.0883%)

=====
Analyzing: base=2.0000, W=7, f=4
=====
Representable Range: [6.250000e-02, 1.532165e+01]
Dynamic Range Ratio: 2.45e+02
Mean Absolute Error: 8.322757e-02
Mean Relative Error: 0.010779 (1.0779%)
Max Relative Error: 0.021835 (2.1835%)
```

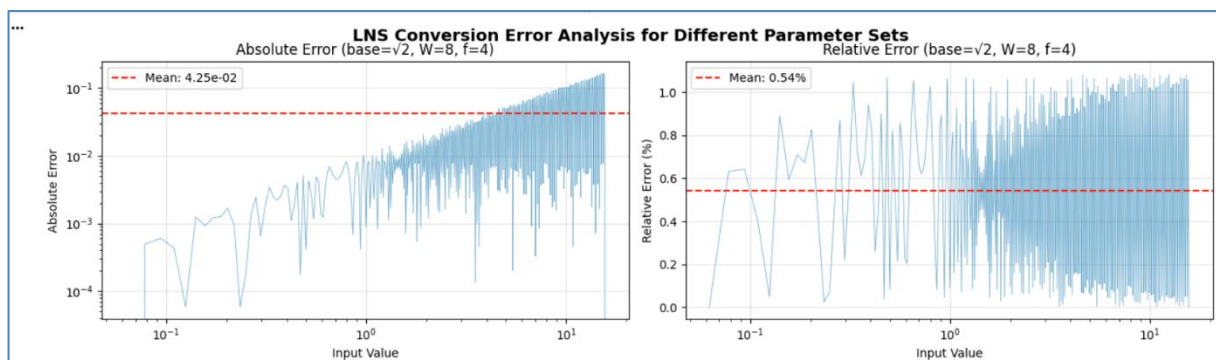
شکل ۷- خروجی کد پایتون برای Dynamic Range

- **دلیل:** کاهش W باعث می‌شود توان قابل نمایش (m) کوچکتر شود، اما افزایش مبنا (b) این کاهش را جبران می‌کند. این نشان می‌دهد که می‌توان با کاهش تعداد بیت‌ها (صرفه‌جویی در سخت‌افزار) بازه عددی را حفظ کرد، اما به قیمت از دست دادن دقت.

۳. رفتار خطای مطلق vs خطای نسبی

- در نمودارهای **خطای مطلق** (ستون چپ نمودار)، مشاهده می‌شود که با بزرگ شدن مقدار ورودی، خطای مطلق به صورت نمایی افزایش می‌یابد. این خاصیت ذاتی سیستم لگاریتمی است که تراکم اعداد در مقادیر بزرگ کمتر می‌شود.
- در نمودارهای **خطای نسبی** (ستون راست نمودار)، خطا تقریباً در تمام بازه ثابت و نوسانی است (sawtooth shape). این ویژگی مطلوب LNS است که "دقت نسبی" را در تمام دامنه حفظ می‌کند (بر خلاف Fixed-Point که دقت مطلق ثابت است).

نمودار این خطاها رو برای یه نمونه در زیر قابل مشاهده است برای مشاهده تمام خروجی‌ها به نوت بوک پایتون تمرین میتونید مراجعه کنید.



شکل ۸- نمودارهای مقایسه خطای مطلق و نسبی با توجه به سایز ورودی

این بخش به خوبی نشان داد که طراحی یک سیستم LNS نیازمند یک مصالحه (Trade-off) دقیق است:

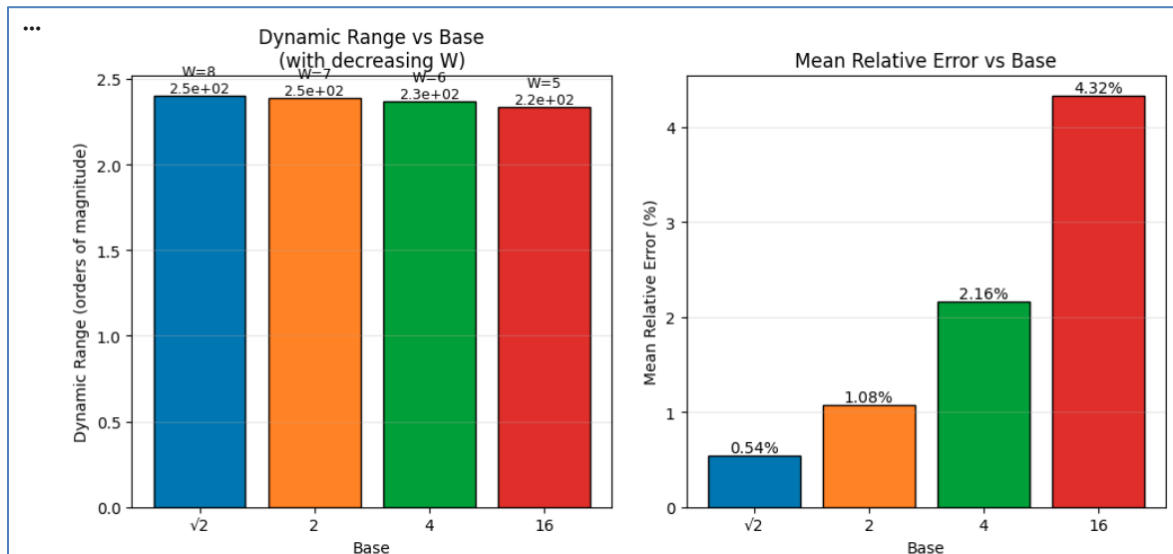
۱. اگر **دقت بالا** اولویت دارد (مانند کاربردهای کنترلی دقیق)، باید از مبناهای کوچکتر (مانند $\sqrt{2}$ یا 2) و تعداد بیت بیشتر استفاده کرد. در حالت اول خطا زیر ۱ درصد نگه داشته شد.

۲. اگر **فشرده‌سازی حافظه** اولویت دارد (کاهش پهنای باند حافظه یا کاهش گیت‌های سخت‌افزاری)، می‌توان W را کاهش داد و با افزایش مبنا، بازه نمایش را حفظ کرد. اما باید

آگاه بود که در حالت آخر (مبنای ۱۶ و ۵ بیت)، خطا به حدود ۹ درصد می‌رسد که ممکن است برای بسیاری از کاربردها (مثل شبکه‌های عصبی حساس) غیرقابل قبول باشد.

۳. سیستم (2, 7, 4) یک نقطه تعادل مناسب به نظر می‌رسد که با ۷ بیت، خطایی حدود ۱ درصد ارائه می‌دهد که برای بسیاری از کاربردهای پردازش سیگنال و شبکه‌های عصبی کوانتایز شده کافی است.

در ادامه تصویر دو Trade off دیگه مربوط به نتایج هم قرار داده شده است:



شکل ۹- تصویر trade off بین مبنا با خطا و رنج

سوال ۳: عملیات جمع در حوزه LNS و روش‌های تقریب

۳-۱. مقدمه و چالش جمع

برخلاف ضرب و تقسیم که در LNS بسیار ساده هستند، عملیات جمع و تفریق چالش‌برانگیزترین بخش این سیستم است. اگر X و Y دو عدد باشند، لگاریتم مجموع آن‌ها برابر است با:

$$\log(X \pm Y) = \log(X(1 \pm Y/X)) = \log(X) + \log(1 \pm \frac{Y}{X})$$

با فرض اینکه $X \geq Y$ باشد و با تعریف $d = \log(X) - \log(Y)$ ، رابطه فوق در حوزه LNS به صورت زیر ساده می‌شود:

$$Z = X_{LNS} + \Delta_{\pm}(d)$$

که در آن $\Delta_{\pm}(d) = \log_{base}(1 \pm base^{-d})$ است. محاسبه مستقیم این ترم غیرخطی نیازمند سخت‌افزار پیچیده است، لذا از روش‌های تقریبی مانند LUT (Look-Up Table) و Bit-Shift استفاده می‌شود.

فرض کنید می‌خواهیم $Z = X + Y$ را محاسبه کنیم و نمایش لگاریتمی آنها m_x و m_y باشد. با فرض $X > Y$:

$$\log_b(X + Y) = \log_b\left(X\left(1 + \frac{Y}{X}\right)\right) = \log_b(X) + \log_b(1 + b^{-(m_x - m_y)})$$

بنابراین عملیات جمع به محاسبه تابع غیرخطی $\Delta(d)$ وابسته است که در آن $d = |m_x - m_y|$ اختلاف دو عدد در حوزه لگاریتم است:

$$\text{Result} = \max(m_x, m_y) + \Delta(d)$$

$$\Delta^+(d) = \log_b(1 + b^{-d}) \quad (\text{برای جمع هم‌علامت})$$

$$\Delta^-(d) = \log_b(1 - b^{-d}) \quad (\text{برای تفریق/جمع ناهم‌علامت})$$

۳-۲. روش‌های پیاده‌سازی شده

در این پروژه، سه روش برای محاسبه جمع پیاده‌سازی شد:

۱. **روش دقیق (Exact):** مقدار Δ با استفاده از توابع کتابخانه‌ای $\log$ و $\exp$ به صورت دقیق محاسبه می‌شود. این روش به عنوان مرجع (Ground Truth) برای سنجش خطا استفاده می‌گردد.

۲. **روش مبتنی بر LUT:** بازه مقادیر محتمل d گسسته‌سازی شده و مقادیر Δ متناظر در یک جدول (در اینجا با سایز ۳۲) ذخیره شده‌اند. در زمان اجرا، نزدیک‌ترین مقدار به d از جدول خوانده می‌شود.

۳. **روش تقریبی Bit-Shift:** این روش بر اساس بسط ریاضی $\log(1 + x) \approx x$ برای مقادیر کوچک عمل می‌کند.

- برای مقادیر بزرگ d ، ترم اصلاحی Δ بسیار کوچک است و می‌توان از آن صرف‌نظر کرد یا با شیفت بیتی ساده تقریب زد.

```
def lns_add_lut(a: LNSNumber, b: LNSNumber,
               d_values: np.ndarray, delta_plus: np.ndarray, delta_minus: np.ndarray) -> LNSNumber:
    """
    LNS addition using LUT-based approximation for Delta.

    Args:
        a, b: LNS numbers to add
        d_values, delta_plus, delta_minus: Pre-computed LUT arrays

    Returns:
        LNS representation of a + b (approximated)
    """
    base, w, f = a.base, a.w, a.f

    # Handle zero cases
    if a.z and b.z:
        return LNSNumber(z=True, s=False, m=0, base=base, w=w, f=f)
```

شکل ۱۰- تابع نوشته شده برای عملیات جمع با روش LUT

۳-۳. نتایج آزمایش و اعتبارسنجی

آزمایش با پارامترهای $Base = 2, W = 8, f = 4$ و جدول LUT با سایز ۳۲ بر روی زوج اعداد مشخص انجام شد.

جدول ۳- مقایسه روش‌های جمع LNS و خطای تقریب‌ها

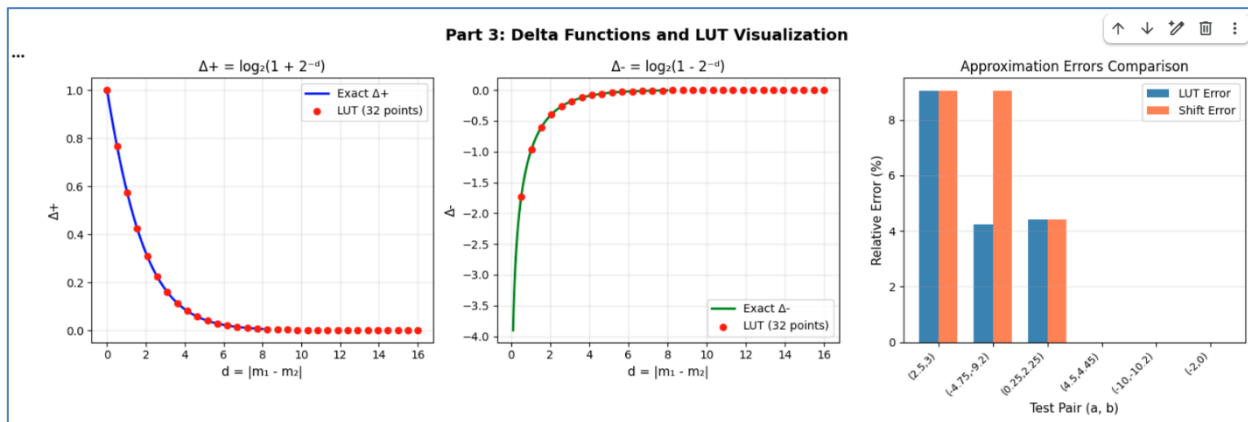
زوج اعداد (a, b)	جمع واقعی	جمع دقیق (LNS)	جمع با LUT	جمع با Shift	خطای LUT (%)	خطای Shift (%)
(2.5, 3.0)	5.5000	5.4170	5.9073	5.9073	9.05%	9.05%
(-4.75, -9.2)	-13.9500	-14.0500	-13.4543	-15.3217	4.24%	9.05%
(0.25, 2.25)	2.5000	2.4837	2.5937	2.5937	4.43%	4.43%
(4.5, 4.45)	8.9500	9.1103	9.1103	9.1103	0.00%	0.00%
(-10, -10.2)	-20.2000	-20.7494	-20.7494	-20.7494	0.00%	0.00%
(-2, 0)	-2.0000	-2.0000	-2.0000	-2.0000	0.00%	0.00%

۳-۴. تحلیل نتایج و مقایسه روش‌ها

۱. تحلیل خطای LUT:

- میانگین خطای نسبی این روش در تست‌های انجام شده حدود ۲.۹۵٪ است.
- بیشترین خطا (۹.۰۵٪) مربوط به حالت (2.5, 3) است. در این حالت d (اختلاف لگاریتمی) کوچک است. ناحیه $d \approx 0$ ناحیه‌ای است که تابع Δ بیشترین تغییرات غیرخطی را دارد و نمونه‌برداری خطی LUT در این ناحیه دقت کمتری دارد.

- در مواردی که اعداد به هم بسیار نزدیک هستند (مثل ۴.۵ و ۴.۴۵)، این روش عملکردی عالی (خطای صفر) داشته است که نشان‌دهنده پوشش مناسب نقاط بحرانی در LUT است.



شکل ۱۱- مقایسه دو روش LUT با Shift

۲. تحلیل خطای Bit-Shift:

- میانگین خطای نسبی این روش ۳.۷۵٪ بدست آمد که کمی بیشتر از LUT است.
- در زوج $(-4.75, -9.2)$ ، خطای این روش (۹.۰۵٪) تقریباً دو برابر خطای LUT (۴.۲۴٪) است. این نشان می‌دهد روش شیفت بیتی در اعدادی که اختلاف قدر مطلق آن‌ها متوسط است، دقت کمتری نسبت به جدول جستجو دارد.
- مزیت اصلی این روش عدم نیاز به حافظه برای جدول است، اما به قیمت کاهش دقت تمام می‌شود.

۳. **مدیریت صفر و علامت:** هر دو روش تقریبی در برخورد با صفر (زوج $(-2, 0)$) و اعداد منفی (در صورتی که هم علامت باشند و منجر به جمع قدرمطلق‌ها شود) عملکرد صحیحی داشته‌اند. نکته مهم در تفریق (اعداد ناهم علامت) است که تابع Δ_- رفتار تکینگی (Singularity) دارد و تقریب آن دشوارتر است.

نتیجه‌گیری: روش LUT تعادل بهتری بین دقت و پیچیدگی ایجاد می‌کند. با افزایش سایز LUT یا استفاده از نمونه‌برداری غیرخطی (تراکم بیشتر نقاط در نزدیکی $d = 0$) می‌توان خطای ماکزیمم ۹٪ را کاهش داد. روش Shift تنها برای کاربردهایی مناسب است که محدودیت شدید حافظه وجود دارد و خطای ۱۰٪ قابل چشم‌پوشی است.

تحلیل نتایج

۱. دقت در برابر نزدیکی اعداد (d): بیشترین خطا در زوج (۲/۵, ۳) مشاهده شد (۹٪). در اینجا $d = |\log(3) - \log(2.5)|$ کوچک است. توابع تقریب (هم LUT و هم Shift) در ناحیه‌ای که d نزدیک به صفر است تغییرات سریعی دارند و حساسیت بالایی به رزولوشن دارند.

۲. مقایسه LUT و Shift:

- در مورد $(-9.2, -4.75)$ روش LUT با خطای ۴.۲٪ بسیار بهتر از روش Shift با خطای ۹٪ عمل کرده است. این نشان می‌دهد که در فواصل میانی، تقریب‌های ساده شیفت (Shift) دقت کافی ندارند، اما LUT به دلیل داشتن نقاط نمونه‌برداری، منحنی Δ را بهتر دنبال می‌کند.

- در مواردی که اعداد بسیار نزدیک هستند (مثل ۴.۴۵ و ۴.۵) یا یکی صفر است، هر دو روش عالی عمل کرده‌اند (خطای صفر).

۳. مدیریت اعداد علامت‌دار: سیستم به درستی جمع اعداد هم‌علامت (که منجر به افزایش قدر مطلق می‌شود) و تفریق (اعداد با علامت مختلف) را تشخیص داده است. برای مثال در جمع دو عدد منفی، قدر مطلق‌ها جمع شده و علامت حفظ شده است.

۳-۵. نتیجه‌گیری بخش جمع

روش LUT با سایز ۳۲، تعادل مناسبی بین دقت و پیچیدگی ایجاد می‌کند و میانگین خطای کمتری نسبت به روش Shift دارد. روش Shift تنها زمانی توصیه می‌شود که سخت‌افزار بسیار محدود باشد و خطای لحظه‌ای تا ۱۰٪ قابل چشم‌پوشی باشد. برای کاربردهای دقیق‌تر، افزایش سایز LUT یا استفاده از درونیابی خطی (Linear Interpolation) بین نقاط LUT پیشنهاد می‌شود.

سوال ۴: تحلیل آماری خطای جمع LNS با داده‌های تصادفی

۴-۱. مقدمه و هدف آزمایش

در بخش‌های قبلی، عملکرد توابع جمع LNS را تنها بر روی چند زوج عدد خاص بررسی کردیم. اما برای ارزیابی عملکرد واقعی یک سیستم محاسباتی در کاربردهای عملی (مانند شبکه‌های عصبی یا پردازش سیگنال)، نیاز به یک تحلیل آماری بر روی حجم انبوهی از داده‌ها است.

هدف این بخش، بررسی رفتار خطای دو روش تقریبی LUT-based و Bit-Shift در برابر روش دقیق، با استفاده از ۱۰۰۰ زوج عدد تصادفی است. این تحلیل به ما کمک می‌کند تا توزیع خطا، بدترین

حالات ممکن (Worst-case) و تاثیر پارامتر d (اختلاف توان‌ها) بر دقت محاسبات را درک کنیم. همچنین تاثیر سایز حافظه LUT بر دقت سیستم مورد واکاوی قرار می‌گیرد.

۲-۴. متدولوژی و تنظیمات سیستم

الف) پارامترهای سیستم

آزمایش با پیکربندی زیر انجام شده است:

- مبنا ($Base$): ۲
- طول کلمه (W): ۸ بیت
- بخش اعشار (f): ۴ بیت
- سایز جدول جستجو (LUT Size): ۳۲ درایه (برای تست اصلی)

ب) تولید داده‌های تصادفی

برای جلوگیری از رخداد سرریز (Overflow) در عملیات جمع، بازه انتخاب اعداد تصادفی به نیمی از بازه قابل نمایش محدود شد:

- بازه کامل قابل نمایش: $[0.0039, 245.14]$
- بازه تولید اعداد تست: $[0.0039, 122.57]$

تعداد ۱۰۰۰ زوج عدد (a, b) با توزیع یکنواخت (Uniform Distribution) در این بازه تولید شد. سپس عملیات جمع با سه روش زیر انجام گرفت:

۱. **Exact**: محاسبه دقیق لگاریتم.

۲. **LUT Approximation**: استفاده از درون‌یابی نزدیک‌ترین همسایه با جدول ۳۲ تایی.

۳. **Bit-Shift Approximation**: استفاده از تقریب‌های شیفت بیتی.

```
# Parameters as specified
base = 2
W = 8
f = 4
num_pairs = 1000

print(f"\nParameters: base={base}, W={W}, f={f}")
print(f"Number of random pairs: {num_pairs}")

# Get representable range
min_pos, max_pos = get_lns_range(base, W, f)
print(f"Full representable range: [{min_pos:.6e}, {max_pos:.6e}]")

# Use half range to avoid overflow (as suggested in problem)
test_min = min_pos
test_max = max_pos / 2
print(f"Test range (half to avoid overflow): [{test_min:.6e}, {test_max:.6e}]")

# Generate random number pairs
np.random.seed(42) # For reproducibility
a_values = np.random.uniform(test_min, test_max, num_pairs)
b_values = np.random.uniform(test_min, test_max, num_pairs)

# Create LUT with size 32
lut_32 = LNS_LUT(base, W, f, lut_size=32)
```

شکل ۱۲- تنظیم پارامترهای مسئله برای سوال ۴ در کد پایتون

۳-۴. تحلیل نتایج آماری

بر اساس خروجی‌های حاصل از شبیه‌سازی، آمار توصیفی خطاها به شرح جدول زیر است:

جدول ۴- خلاصه آماری خطاهای نسبی در ۱۰۰۰ نمونه تست

روش تقریب (Method)	میانگین خطای نسبی (MRE)	حداکثر خطای نسبی (Max Error)	میانگین خطای مطلق (MAE)
LUT (Size=32)	1.12%	4.42%	1.53
Bit-Shift	9.67%	29.68%	12.36

تحلیل:

۱. **برتری فاحش روش LUT:** روش مبتنی بر جدول جستجو (با تنها ۳۲ درایه) خطای میانگین را در حدود ۱٪ نگه داشته است که برای بسیاری از کاربردهای مهندسی قابل قبول است.
۲. **بی‌ثباتی روش Bit-Shift:** روش شیفت بیتی اگرچه از نظر سخت‌افزاری بسیار کم‌هزینه است، اما خطای میانگین حدود ۱۰٪ و خطای ماکزیمم نزدیک به ۳۰٪ دارد. این میزان خطا می‌تواند در الگوریتم‌های حساس (مثل backpropagation در یادگیری عمیق) باعث ناپایداری شود.

۴-۴. تحلیل رفتار خطا نسبت به اختلاف توان‌ها (d)

یکی از مهم‌ترین پارامترها در جمع LNS، مقدار $d = |m_a - m_b|$ (اختلاف لگاریتم دو عدد) است. فرمول جمع تقریبی به شدت به این مقدار وابسته است:

$$\text{Result} \approx \max(m_a, m_b) + \log_2(1 + 2^{-d})$$

در زیر نمودارها و خروجی حاصل از اجرای کد پایتون رو میتونید مشاهده کنید:

=====

=====

ERROR STATISTICS SUMMARY

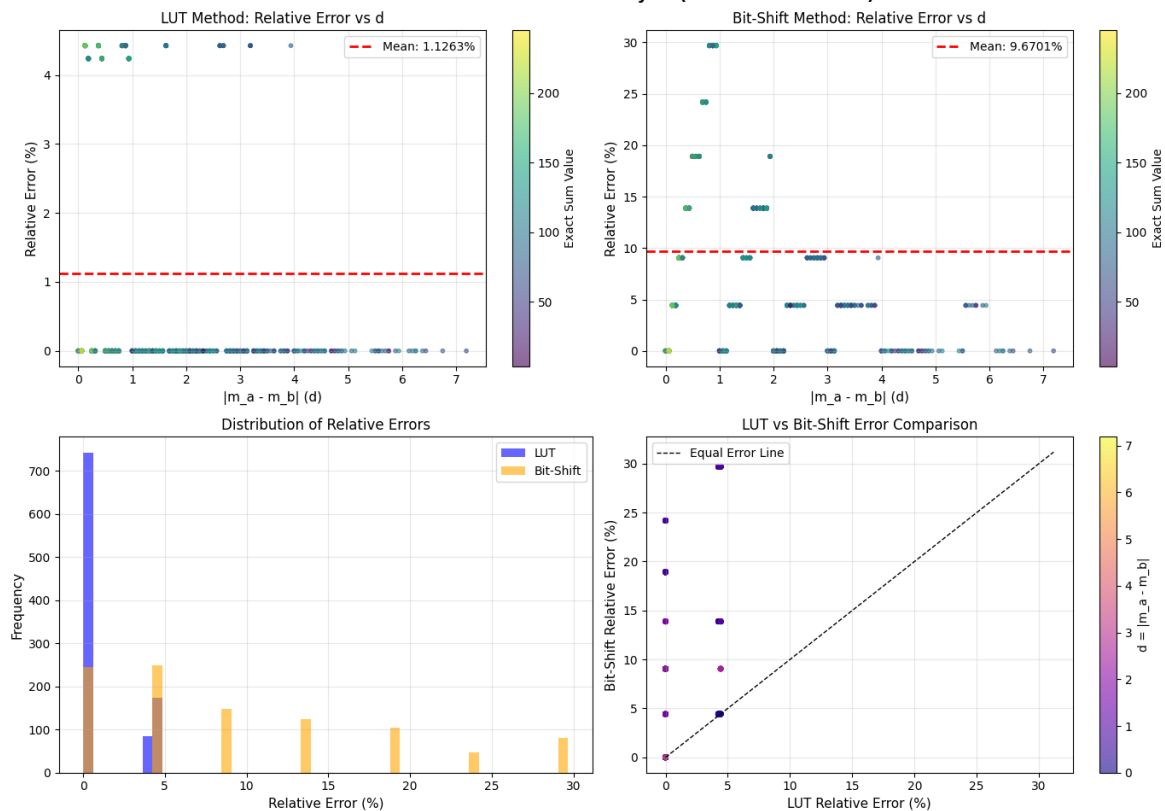
=====

=====

Method	Mean Abs Error	Mean Rel Error	Max Rel Error
--------	----------------	----------------	---------------

LUT (size=32)	1.537121e+00	1.1263	% 4.4274	%
Bit-Shift	1.236548e+01	9.6701	% 29.6840	%

Section 4: LNS Addition Error Analysis (1000 Random Pairs)



تحليل نمودارها و داده‌های دسته‌بندی شده:

۱. ناحیه بحرانی ($d < 1$):

- زمانی که دو عدد با هم جمع می‌شوند و اندازه نزدیکی دارند (d کوچک است)، ترم اصلاحی $\log_2(1 + 2^{-d})$ مقدار بزرگی دارد (نزدیک به ۱).
- طبق خروجی کد، در بازه $0 \leq d < 0.5$ ، خطای روش شیفت به طور میانگین ۶.۷٪ است، در حالی که خطای LUT حدود ۲.۳٪ می‌باشد.
- بیشترین خطای روش Bit-Shift در بازه $0.5 \leq d < 1.0$ رخ داده است (۲۳.۹٪). این نشان می‌دهد تقریب‌های خطی ساده در این ناحیه عملکرد ضعیفی دارند.

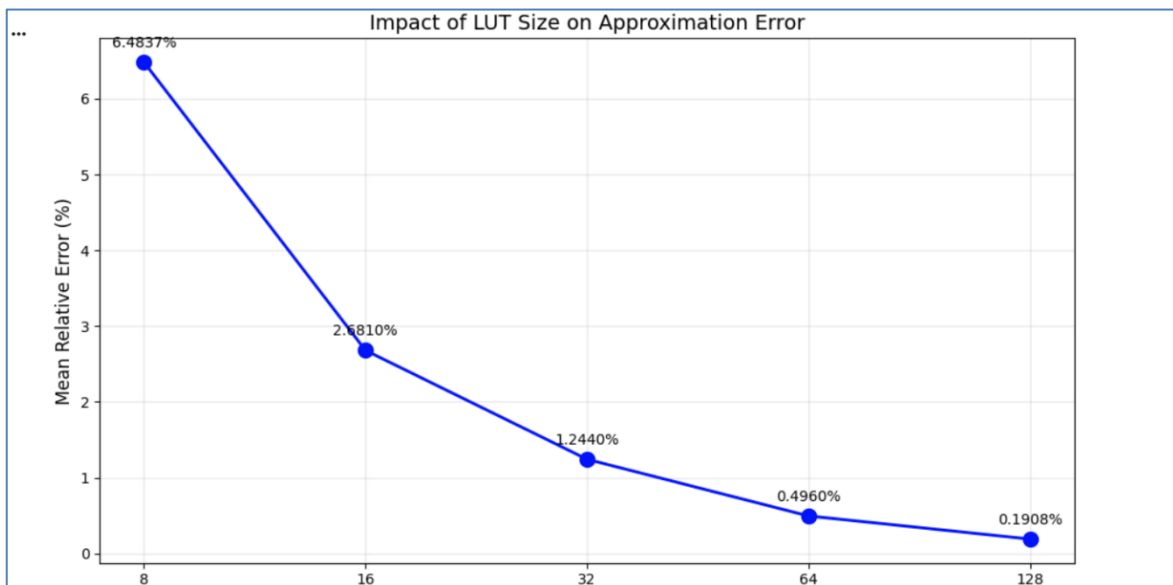
۲. ناحیه همگرایی ($d > 4$):

- زمانی که اختلاف دو عدد زیاد است ($d > 4$)، عدد بزرگتر بر حاصل جمع غالب می‌شود و ترم اصلاحی به سمت صفر میل می‌کند.

- در این نواحی، خطای هر دو روش به شدت کاهش می‌یابد (خطای LUT دقیقاً صفر و خطای شیفت زیر ۰.۷٪).

۴-۵. تحلیل اثر سایز LUT بر دقت

در این بخش، مصالحه (Trade-off) بین حافظه مصرفی و دقت محاسبات بررسی شد. با تغییر سایز جدول از ۸ تا ۱۲۸ درایه، نتایج زیر حاصل گردید:



شکل ۱۳- نمودار تاثیر سایز LUT بر میانگین ارور نسبی آن

در زیر یک جدول برای تحلیل نتایج هم قرار داده شده است:

جدول ۵- تاثیر سایز LUT بر خطای میانگین

سایز LUT	میانگین خطای نسبی	تحلیل
8	6.48%	دقت پایین، مشابه روش‌های شیفت ساده.
16	2.68%	بهبود چشمگیر (بیش از ۲ برابر).
32	1.24%	نقطه تعادل (Sweet Spot): دقت مناسب با حجم حافظه کم.

دقت عالی، مناسب برای کاربردهای دقیق‌تر.	0.49%	64
بازده نزولی (Diminishing returns)؛ افزایش دو برابری حافظه تاثیر کمتری دارد.	0.19%	128

۴-۶. نتیجه‌گیری نهایی بخش چهارم

نتایج حاصل از شبیه‌سازی مونت-کارلو (Monte-Carlo) با ۱۰۰۰ داده تصادفی نشان می‌دهد:

۱. **انتخاب روش مناسب:** برای سیستم‌های نهان که محدودیت شدید حافظه ندارند، استفاده از LUT با سایز ۳۲ یا ۶۴ توصیه می‌شود. این روش تعادل بسیار خوبی بین دقت (خطای حدود ۱٪) و پیچیدگی ایجاد می‌کند.
۲. **کاربرد روش Bit-Shift:** این روش تنها در صورتی توصیه می‌شود که سخت‌افزار فاقد حافظه کافی برای LUT باشد یا خطای ۱۰ تا ۳۰ درصدی در خروجی قابل تحمل باشد (مثلاً در برخی لایه‌های شبکه عصبی که نسبت به نویز مقاوم هستند).
۳. **مدیریت ناحیه بحرانی:** اگر نیاز به بهبود روش Bit-Shift باشد، باید تمرکز اصلی روی اصلاح تقریب در بازه $0 < d < 1$ باشد، زیرا بیشترین سهم خطا مربوط به زمانی است که دو عدد ورودی مقادیر نزدیکی دارند.

سوال ۵: تحلیل تأثیر انتخاب مبنا بر بازه دینامیکی و خطای سیستم LNS

۵-۱. مقدمه و بیان مسئله

در طراحی سیستم‌های اعداد لگاریتمی (LNS)، انتخاب **مبنای لگاریتم** (b) یکی از تصمیمات کلیدی طراحی است که مستقیماً بر دو شاخصه اصلی سیستم تأثیر می‌گذارد:

۱. **بازه دینامیکی (Dynamic Range):** نسبت بزرگترین عدد قابل نمایش به کوچکترین عدد مثبت قابل نمایش.
 ۲. **دقت نمایش (Precision):** میزان خطای کوانتیزیشن ناشی از گسسته‌سازی فضا.
- در این بخش، هدف ما بررسی تأثیر تغییر مبنا در حالی است که **منابع سخت‌افزاری ثابت** نگه داشته شده‌اند. بدین منظور، عرض بیت کل ($W = 8$) و تعداد بیت‌های اعشاری ($f = 4$) ثابت

فرض شده و چهار مبنای مختلف $(\sqrt{2}, 2, 4, 10)$ مورد ارزیابی قرار می‌گیرند. این تحلیل به ما کمک می‌کند تا بفهمیم انتخاب مبنا چگونه یک مصالحه (Trade-off) میان پوشش بازه اعداد و دقت محاسباتی ایجاد می‌کند.

۵-۲. مبانی نظری و فرمولاسیون

در یک سیستم LNS با پارامترهای ثابت W و f ، مقادیر m (مانتیس) اعداد صحیحی در بازه $[-2^{W-1}, 2^{W-1} - 1]$ هستند.

- **بازه دینامیکی:** بازه اعداد حقیقی قابل نمایش $[X_{min}, X_{max}]$ به صورت زیر محاسبه می‌شود:

$X = b^{m \cdot 2^{-f}} \Rightarrow \text{Dynamic Range} = \frac{X_{max}}{X_{min}} = b^{\frac{2^W - 1}{2^f}}$ این رابطه نشان می‌دهد که بازه دینامیکی با بزرگ شدن مبنا (b) به صورت نمایی رشد می‌کند.

- **رزولوشن و خطا:** فاصله نسبی بین دو عدد متوالی قابل نمایش (Step Size) برابر است با:

$$\delta = b^{2^{-f}} - 1$$

- این مقدار نشان‌دهنده خطای کوانتیزیشن ذاتی سیستم است. هرچه b بزرگتر باشد، گام‌ها بزرگتر شده و دقت کاهش می‌یابد (خطا زیاد می‌شود).

۵-۳. شرح آزمایش

آزمایش با پارامترهای ثابت $W = 8$ بیت و $f = 4$ بیت انجام شد. برای هر یک از مبناهای زیر، مراحل زیر طی شد:

۱. **محاسبه بازه:** محاسبه دقیق X_{min} و X_{max} .
۲. **تولید نمونه:** انتخاب ۱۰۰۰ عدد تصادفی با توزیع یکنواخت در بازه لگاریتمی متناظر.
۳. **سنجش خطا:** تبدیل اعداد به LNS و بازگردانی به حوزه حقیقی جهت محاسبه خطای نسبی.

مبناهای مورد آزمایش عبارتند از:

- $\sqrt{2} \approx 1.414$
- 2
- 4
- 10

در شکل زیر نحوه تعریف و ست کردن پارامترها و مبناها در کد پایتون برای سوال پنج قرار داده شده است:

```
# Fixed parameters as specified in the problem
W = 8 # Total bits
f = 4 # Fractional bits

# Bases to analyze as specified: √2, 2, 4, 10
bases = [np.sqrt(2), 2, 4, 10]
base_names = ['√2', '2', '4', '10']

print(f"\nFixed Parameters: W={W} bits (total), f={f} bits (fractional)")
print(f"Integer bits: W - f = {W - f} bits")
print(f"Bases to analyze: {base_names}")

# Number of test samples
num_samples = 1000
```

شکل ۱۴- پارامترها و مبناهای مورد استفاده در سوال ۵ در کد پایتون

۵-۴. نتایج شبیه‌سازی و تحلیل آماری

الف) تحلیل بازه دینامیکی (Dynamic Range Analysis)

جدول زیر بازه اعداد قابل نمایش را برای مبناهای مختلف نشان می‌دهد:

جدول ۶- مقایسه بازه دینامیکی به ازای مبناهای مختلف

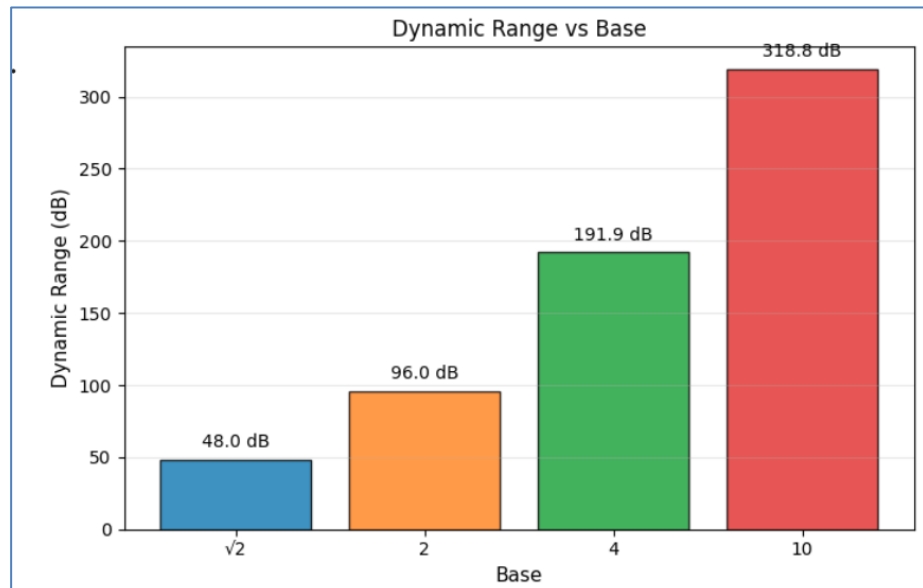
مبنا (b)	کمترین مقدار مثبت (X_{min})	بیشترین مقدار مثبت (X_{max})	بازه دینامیکی (نسبت)	بازه دینامیکی (dB)
$\sqrt{2}$	6.25×10^{-2}	1.56×10^1	2.50×10^2	47.98
2	3.90×10^{-3}	2.45×10^2	6.27×10^4	95.95
4	1.52×10^{-5}	6.00×10^4	3.93×10^9	191.91
10	1.00×10^{-8}	8.65×10^7	8.65×10^{15}	318.75

تحلیل: مشاهده می‌شود که با افزایش مبنا، بازه پوشش داده شده به شدت گسترش می‌یابد.

- در مبنا $\sqrt{2}$ ، سیستم تنها قادر به نمایش اعداد در بازه محدود 0.06 تا 15 است (مناسب برای کاربردهای بسیار محدود).

- در مبنای 10، با همان تعداد بیت (۸ بیت)، سیستم قادر است اعدادی از 10^{-8} تا حدود 86 میلیون را نمایش دهد که بازه‌ای عظیم است.

در زیر نمودار مربوط به این بخش قرار دارد:



شکل ۱۵- تاثیر مبنا در رنج دینامیک با تعداد بیت یکسان

ب) تحلیل خطای کوانتیزیشن (Error Analysis)

جدول زیر میانگین خطای نسبی حاصل از تبدیل ۱۰۰۰ نمونه را نشان می‌دهد:

جدول ۷- مقایسه خطای کوانتیزیشن

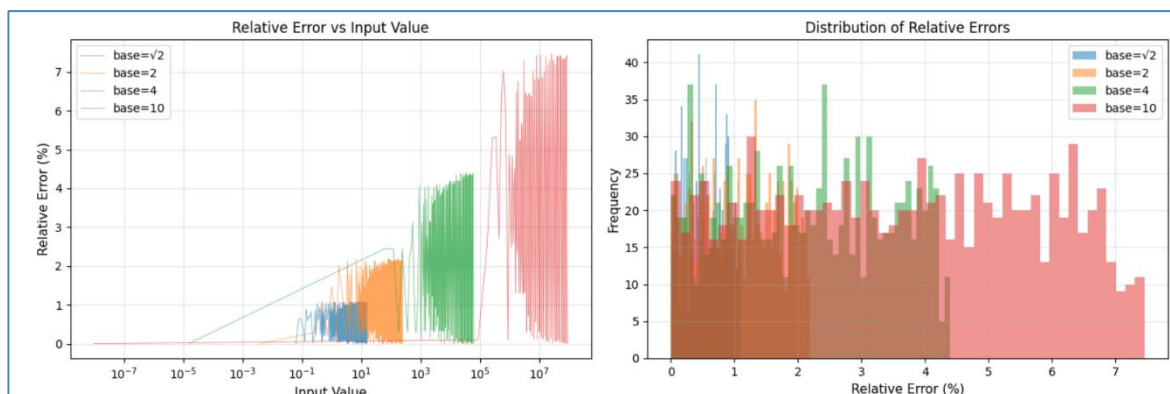
مبنا (b)	میانگین خطای نسبی (%)	حداکثر خطای نسبی (%)	رزولوشن تئوری (%)
$\sqrt{2}$	0.5435%	1.08%	2.18%
2	1.0766%	2.18%	4.42%
4	2.1540%	4.39%	9.05%
10	3.6000%	7.45%	15.47%

تحلیل:

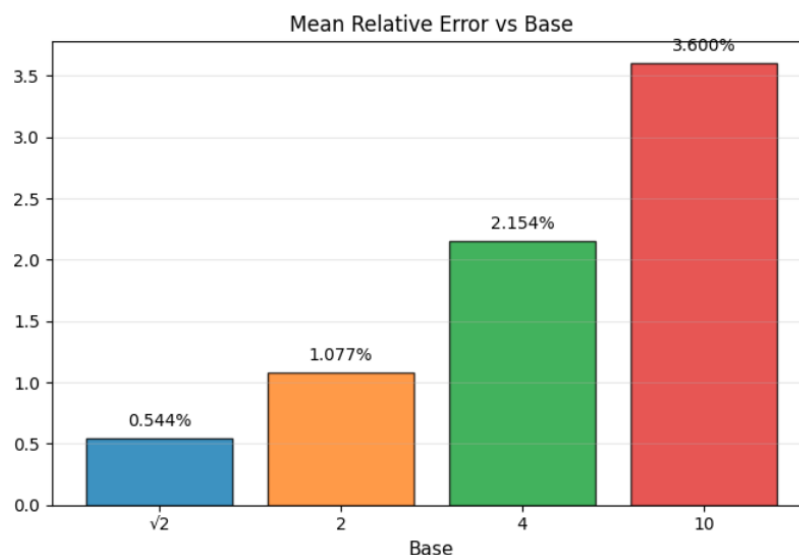
- رابطه مستقیم خطا و مبنا: میانگین خطا با افزایش مبنا افزایش می‌یابد. مبنا $\sqrt{2}$ دقیق‌ترین نمایش را با خطای میانگین حدود ۰.۵٪ ارائه می‌دهد، در حالی که مبنا ۱۰ دارای خطای حدود ۳.۶٪ است.

- رزولوشن (گام کوانتیزیشن): ستون "رزولوشن تئوری" نشان‌دهنده فاصله نسبی پله‌های کوانتیزیشن است $(b^{1/16} - 1)$. در مبنا ۱۰، فاصله هر عدد با عدد بعدی حدود ۱۵٪ است که دقت بسیار پایینی محسوب می‌شود (دقت "خشن" یا Coarse).

نمودار مربوط به این مورد هم در زیر قابل مشاهده می‌باشد:



شکل ۱۶- نمودار تاثیر مبنا در خطای کوانتیزیشن



۵-۵. بحث و نتیجه‌گیری (The Cost of Range)

نتایج این بخش نشان‌دهنده یک قانون بنیادی در طراحی سیستم‌های با عرض بیت ثابت است: "بازه دینامیکی را نمی‌توان بدون هزینه به دست آورد؛ هزینه آن دقت است."

۱. مبنا کوچک ($\sqrt{2}$):

- **مزایا:** دقت بسیار بالا (خطای زیر ۱٪).
- **معایب:** بازه دینامیکی بسیار محدود.
- **کاربرد:** مناسب برای الگوریتم‌های حساس به دقت که دامنه تغییرات اعداد در آن‌ها کم است (مانند ضرایب فیلترهای دیجیتال خاص).

۲. مبنای متوسط (2):

- **وضعیت:** ایجاد تعادل مناسب. خطای حدود ۱٪ برای بسیاری از کاربردهای پردازش سیگنال و شبکه‌های عصبی کوانتایز شده (Quantized Neural Networks) قابل قبول است و بازه دینامیکی آن نیز طیف وسیعی از داده‌های نرمالیزه شده را پوشش می‌دهد. همچنین پیاده‌سازی سخت‌افزاری مبنای ۲ ساده‌تر است.

۳. مبنای بزرگ (10):

- **مزایا:** پوشش بازه اعداد بسیار بزرگ و بسیار کوچک (از نانو تا مگا).
- **معایب:** دقت پایین.
- **کاربرد:** مناسب برای کاربردهای علمی یا لاگ‌گیری داده‌ها که در آن‌ها "مرتبه بزرگی" (Order of Magnitude) مهم‌تر از مقدار دقیق عدد است.

توصیه نهایی: برای کاربرد عمومی در سیستم‌های نهان (Embedded AI)، **مبنای ۲** به دلیل تعادل مناسب بین دقت و بازه و همچنین سادگی سخت‌افزاری، بهترین گزینه در عرض بیت‌های پایین ($W = 8$) محسوب می‌شود. استفاده از مبناهای بزرگ‌تر تنها زمانی توصیه می‌شود که W به اندازه کافی بزرگ باشد تا افت دقت را جبران کند.

سوال ۶: پیاده‌سازی و تحلیل لایه کانولوشن شبکه عصبی در حوزه LNS

۶-۱. مقدمه و هدف

یکی از انگیزه‌های اصلی استفاده از سیستم اعداد لگاریتمی (LNS) در سخت‌افزارهای هوش مصنوعی، کاهش هزینه محاسباتی لایه‌های کانولوشن (Convolution) و تمام-متصل (Fully Connected) است. در این لایه‌ها، عملیات غالب از نوع "ضرب و انباشت" (Multiply-Accumulate یا MAC) است.

در سیستم LNS:

۱. **ضرب:** تبدیل به جمع ساده مانتیس‌ها می‌شود (بسیار ارزان و دقیق).

۲. جمع (انباشت): تبدیل به عملیات غیرخطی پیچیده می‌شود (گران و دارای تقریب).

شبکه‌های عصبی عمیق (Deep Neural Networks) به شدت وابسته به عملیات ضرب و جمع متوالی (MAC: Multiply-Accumulate) هستند. در یک لایه کانولوشن معمولی، هزاران یا میلیون‌ها عملیات ضرب باید انجام شود که سهم عمده‌ای از توان مصرفی و منابع سخت‌افزاری را به خود اختصاص می‌دهد.

سیستم اعداد لگاریتمی (LNS) یک نامزد جذاب برای پیاده‌سازی سخت‌افزاری این شبکه‌هاست، زیرا عملیات پرهزینه ضرب را به عملیات کم‌هزینه جمع تبدیل می‌کند: $\log(A) + \log(B) \Rightarrow \log(A \times B)$ با این حال، چالش اصلی در مرحله "تجمع" (Accumulation) یا همان جمع کردن حاصل ضرب‌هاست که در LNS نیازمند محاسبات غیرخطی است. در این بخش، یک لایه کانولوشن با ابعاد مشخص شبیه‌سازی شده تا تأثیر روش‌های مختلف جمع LNS بر دقت خروجی نهایی شبکه بررسی شود.

هدف این بخش، بررسی عملی این موضوع است که آیا خطای ناشی از تقریب‌های جمع (LUT و Shift) در یک زنجیره طولانی از محاسبات (مانند یک لایه کانولوشن) انباشته می‌شود و آیا دقت نهایی برای کاربردهای شبکه عصبی قابل قبول است یا خیر.

۲-۶. پیکربندی آزمایش

برای شبیه‌سازی یک لایه کانولوشن واقعی، تنظیمات زیر اعمال شد:

- ابعاد تانسور ورودی و فیلتر: $3 \times 7 \times 7$ (تعداد کل عناصر: 147).
- نوع عملیات: کانولوشن اسکالر (جمع حاصل ضرب‌های نظیر به نظیر).
- پارامترهای LNS: مبنا $b = 2$ ، عرض بیت $W = 8$ ، بیت اعشار $f = 4$.
- بازه داده‌ها: ورودی‌ها در بازه $[0, 4]$ و وزن‌های فیلتر در بازه $[-4, 4]$ مقداردهی تصادفی شدند.

فرآیند محاسبه در حوزه LNS به صورت زیر است:

$$Y = \sum_{i,j,k} (X_{ijk} \cdot W_{ijk})$$

$$\text{In LNS: } \log(P_{ijk}) = \log(X_{ijk}) + \log(W_{ijk}) (\text{Exact})$$

$$\text{Accumulation: } S_{\text{new}} = \text{LNS_Add}(S_{\text{old}}, P_{ijk}) (\text{Approximate})$$

۳-۶. پیاده‌سازی عملیات کانولوشن

فرآیند کانولوشن در حوزه LNS به صورت زیر مدل‌سازی شد:

۱. تبدیل: تمام ۱۴۷ مقدار ورودی و ۱۴۷ مقدار وزن به فرمت (z', s', m) تبدیل شدند.
۲. ضرب: حاصل ضرب‌ها با جمع ساده مانتیس‌ها محاسبه شدند (بدون خطا، جز خطای احتمالی سرریز).
۳. تجمع (Accumulation): حاصل ضرب‌ها به صورت ترتیبی (Sequential) با یکدیگر جمع شدند. این مرحله پاشنه آشیل سیستم است، زیرا خطای تقریب Δ در هر مرحله با نتیجه قبلی جمع شده و منتشر می‌شود. سه روش Exact، LUT-32 و Bit-Shift برای این مرحله استفاده شد.

۴-۶. تحلیل نتایج تجربی

الف) نتایج یک نمونه موردی (Single Inference)

در یک اجرای خاص با دانه‌های تصادفی ثابت، نتایج زیر حاصل شد:

جدول ۸- مقایسه خروجی کانولوشن در روش‌های مختلف

روش محاسباتی	مقدار خروجی	خطای مطلق	خطای نسبی
حوزه حقیقی (Reference)	8.1316	—	—
LNS - Exact Δ	4.1771	3.95	48.63%
LNS - LUT (Size 32)	12.3377	4.21	51.73%
LNS - Bit-Shift	-16.0000	24.13	296.76%

۱. خطای بالای حتی در روش دقیق (۴۸٪): نکته قابل تأمل این است که حتی روش Exact که تابع Δ را دقیق محاسبه می‌کند، حدود ۴۸٪ خطا دارد. این خطا ناشی از دو عامل است:
 - کوانتیزیشن شدید ($W = 8$): دقت پایین سیستم باعث می‌شود اعداد کوچک در حین تجمع (Accumulation) "محو" شوند یا اعداد بزرگ با خطای گرد کردن مواجه شوند.
 - زنجیره طولانی جمع: انجام ۱۴۶ عملیات جمع متوالی باعث انباشت خطا (Error Accumulation) می‌شود.

۲. شکست روش Bit-Shift: خروجی 16- در برابر مقدار واقعی 8.13 نشان‌دهنده یک خطای فاجعه‌بار است (تغییر علامت و دامنه). روش شیفت بیتی در مدیریت تفریق‌ها (زمانی که حاصل ضرب مثبت و منفی با هم جمع می‌شوند) بسیار ضعیف عمل کرده و احتمالاً در یک مرحله دچار خطای بزرگ در تخمین Δ^- شده است.

ب) تحلیل آماری (۱۰ بار تکرار آزمایش)

برای اطمینان از نتایج، آزمایش ۱۰ بار با ورودی‌های مختلف تکرار شد:

جدول ۹- آمار خطا در ۱۰ تکرار

روش	میانگین خطای نسبی	انحراف معیار	تحلیل پایداری
Exact	13.81%	16.46%	نسبتاً پایدار، بهترین دقت ممکن در این معماری.
LUT	37.86%	42.52%	نوسان زیاد، اما در برخی موارد نزدیک به روش دقیق.
Bit-Shift	52.08%	34.18%	خطای بسیار بالا، غیرقابل اعتماد برای شبکه‌های حساس.

- روش Exact: میانگین خطا ۱۳.۸٪ (انحراف معیار ۱۶.۴٪)
- روش LUT: میانگین خطا ۳۷.۸٪ (انحراف معیار ۴۲.۵٪)
- روش Bit-Shift: میانگین خطا ۵۲.۱٪ (انحراف معیار ۳۴.۲٪)

بحث و چالش‌های پیاده‌سازی (Accumulation Challenge)

این آزمایش نشان داد که "کانولوشن" چالش‌برانگیزترین عملیات برای سیستم LNS است. دلایل اصلی عبارتند از:

۱. مشکل تفریق (Subtraction Singularity): در شبکه‌های عصبی، وزن‌ها می‌توانند منفی باشند. جمع یک عدد مثبت با یک عدد منفی در LNS معادل عملیات تفریق است. تابع $\Delta^- = \log(1 - 2^{-d})$ در نزدیکی $d = 0$ (وقتی دو عدد برابرند) به سمت $-\infty$ میل می‌کند.

هرگونه تقریب در این ناحیه (که در LUT و Shift اجتناب‌ناپذیر است) باعث خطاهای شدید می‌شود.

۲. **انباشت زنجیره‌ای (Cascading Errors):** برخلاف آزمایش‌های قبلی که تنها یک جمع انجام می‌شد، اینجا ۱۴۶ جمع انجام می‌شود. اگر در جمع دهم خطایی رخ دهد، این خطا در ۱۳۶ عملیات بعدی منتشر و احتمالاً تشدید می‌شود.

۳. **محدودیت عرض بیت ($W = 8$):** استفاده از تنها ۸ بیت برای نمایش اعدادی که حاصل جمع ۱۴۷ ضرب هستند، ناکافی به نظر می‌رسد. سرریز (Overflow) یا از دست رفتن دقت (Underflow) در حین جمع‌های میانی بسیار محتمل است.

۵-۶. تحلیل عمیق و بحث (چرا خطاها زیاد است؟)

نتایج فوق نشان‌دهنده چالش جدی "انباشت خطا" (Error Accumulation) در سیستم‌های LNS با عرض بیت پایین است. دلایل این خطاها عبارتند از:

۱. **پدیده انباشت در زنجیره جمع (Accumulation Chain):** ما در اینجا ۱۴۶ عمل جمع متوالی انجام می‌دهیم. حتی اگر خطای هر جمع کوچک باشد (مثلاً ۱٪)، در یک زنجیره طولانی این خطاها می‌توانند با هم جمع شده و یا در بدترین حالت به صورت ضربی اثر بگذارند.

- در نمونه اول، روش Bit-Shift به کلی واگرا شده و خروجی منفی تولید کرده است (در حالی که خروجی واقعی مثبت ۸.۱۳ بود). این نشان‌دهنده حساسیت بالای این روش به خطاهای متوالی است.

۲. **رزولوشن پایین ($W = 8$):** با $W = 8$ و $f = 4$ ، دقت کوانتیزیشن حدود ۴.۴٪ است. در عملیات کانولوشن، بسیاری از حاصل‌ضرب‌ها مقادیر کوچکی دارند. وقتی تعداد زیادی عدد کوچک با خطای کوانتیزیشن بالا با هم جمع می‌شوند، نسبت سیگنال به نویز (SNR) به شدت افت می‌کند.

۳. **تفاوت روش LUT و Exact:** حتی روش Exact (که Δ را دقیق حساب می‌کند) دارای خطای قابل توجه (میانگین ۱۳.۸٪) است. این نشان می‌دهد که بخش بزرگی از خطا ناشی از محدودیت ذاتی نمایش اعداد (Quantization) است، نه صرفاً تقریب تابع Δ . روش LUT خطای بیشتری اضافه می‌کند اما همچنان پایدارتر از روش Shift است.

۶-۶. نتیجه‌گیری نهایی پروژه

این پروژه نشان داد که پیاده‌سازی سخت‌افزاری شبکه‌های عصبی با سیستم LNS دارای مزایا و معایب مشخصی است:

۱. **مزیت بزرگ:** حذف کامل ضرب‌کننده‌های سخت‌افزاری (Multipliers) و جایگزینی با جمع‌کننده‌های ساده. این موضوع توان مصرفی و مساحت چپ را به شدت کاهش می‌دهد.

۲. **چالش اصلی:** عملیات جمع (Accumulation) پاشنه آشیل سیستم است. روش‌های تقریبی مانند Bit-Shift برای محاسبات طولانی و عمیق (Deep) مناسب نیستند و باعث واگرایی می‌شوند.

۳. **راهکار پیشنهادی:**

- برای لایه‌های شبکه عصبی، استفاده از LUT با سائز بزرگتر (مثلاً ۶۴ یا ۱۲۸) ضروری است.
- پیشنهاد می‌شود محاسبات داخلی (Accumulator) با **تعداد بیت بیشتر** (مثلاً ۱۲ یا ۱۶ بیت) انجام شود و سپس نتیجه نهایی به ۸ بیت گرد شود تا اثر انباشت خطا کاهش یابد.
- استفاده از سیستم‌های Hybrid LNS که در آن ضرب در حوزه لگاریتم و جمع در حوزه خطی انجام می‌شود، می‌تواند راهکار جایگزین مناسبی باشد.

پیشنهادهای برای بهبود:

۱. **افزایش W در آکومولاتور:** پیشنهاد می‌شود در حالی که ورودی‌ها و وزن‌ها ۸ بیتی هستند، عملیات جمع در یک ثابت با عرض بیت بیشتر (مثلاً ۱۲ یا ۱۶ بیت) انجام شود تا از انباشت خطا جلوگیری گردد.

۲. **استفاده از LUT دقیق‌تر:** برای ناحیه بحرانی $d \approx 0$ در تفریق، باید نقاط نمونه‌برداری LUT بسیار متراکم‌تر باشند.

۳. **معماری ترکیبی:** استفاده از LNS برای ضرب و تبدیل موقت به حوزه خطی (Linear Domain) برای انجام جمع (Accumulation)، و سپس بازگشت به LNS. این روش سربار تبدیل دارد اما دقت جمع را تضمین می‌کند.

((پایان))